

Instructions

1. This is an individual assignment. The submission **deadline is 23:59 April 9, 2026 HKT**.
2. **Fill in** this PDF form using Adobe Acrobat Reader. Do **NOT** handwrite, use a tablet/iPad to write, or add new text boxes. Handwritten or scanned submissions will **NOT** be graded.
3. When saving, use **File** → **Save** (not “Print to PDF”). Rename the file to *studentID.pdf* (e.g., *1155200000.pdf*). Using “Print” or “Print to PDF” will flatten the form fields, making your submission **unreadable**.
4. Submit this homework to Blackboard before the deadline. You have at most 3 attempts.

Your Name:

Your Student ID Number:

1. (8 points) Same-Origin Policy

Select the difference between the origins of the URLs below and the origin of `http://one.example.com/dir/page.html`. If there is no difference, select “No difference”.

- `http://example.com/dir2/page.html`

Different host ☐ Different protocol ☐ Different port ☐ No difference ☐

- `http://one.example.com:81/dir/ect.html`

Different host ☐ Different protocol ☐ Different port ☐ No difference ☐

- `http://one.example.com/dir/inner/another.html`

Different host ☐ Different protocol ☐ Different port ☐ No difference ☐

- `http://two.example.com/secure.html`

Different host ☐ Different protocol ☐ Different port ☐ No difference ☐

2. (24 points) Code Review: Vulnerability Analysis

The `hw2-q2/` directory contains a bookstore web application built with PHP and SQLite. The application includes user authentication, book management, and search functionality. The following files are included:

- `config.php` – Database initialization and security helper functions
- `login.php` – User authentication (login/logout)
- `index.php` – Main page with book listing and add-book form
- `search.php` – Book search with year-range filtering
- `addbook.php` – Handler for adding new books
- `admin.php` – Admin panel for managing (deleting) books
- `style.css` – Stylesheet (not relevant for security analysis)

The developer has applied proper security measures in most areas (CSRF tokens, prepared statements, output encoding, etc.), but the code still contains three security vulnerabilities. Your task is to find all three.

For each vulnerability, state its type, explain why it is a vulnerability, and describe how to fix it.

Additionally, fix all three vulnerabilities in the code and submit a unified diff file named `q2.diff`. **The diff file is mandatory — no diff file means zero mark for this question.** To generate the diff file, first make a copy of the original `hw2-q2/` directory (e.g., `cp -r hw2-q2 hw2-q2-fixed`), then apply your fixes to the copy, and run:

```
diff -ruN hw2-q2/ hw2-q2-fixed/ > q2.diff
```

a) (8 points) **Vulnerability 1**

Vulnerability type:

Why is it a vulnerability?

How to fix:

b) (8 points) **Vulnerability 2**

Vulnerability type:

Why is it a vulnerability?

How to fix:

c) (8 points) **Vulnerability 3**

Vulnerability type:

Why is it a vulnerability?

How to fix:

3. (18 points) SQL Injection

Consider the following PHP script for authenticating a remote user.

```
$uname = $_GET["user"];
$password = $_GET["password"];
$sql = "SELECT * FROM users
        WHERE username='$uname' AND password = '$password'";
$result = $db->query($sql);
if($result->num_rows > 0) {
    echo "Hello '$uname'";
}
else{
    echo "Login failed";
}
```

- a) (6 points) Give an example of input that can successfully subvert the above authentication code and login as user *admin* without knowing the password. Your answer should include *username* and *password* you design.

\$uname =

\$password =

- b) (6 points) Give an example of input that can delete table *users*.

\$uname =

\$password =

- c) (6 points) Some developers suggest simply escaping all single quotes with a backslash to identify these single quotes from users; for example, user input "tom'" becomes "tom\'" on the server side. Give an example to bypass this single quote escaping and achieve the goal of problem 3.a. (Hint: Check Table 11.1 Special Character Escape Sequences in MySQL.)

\$uname =

\$password =

4. (26 points) XSS Attacks and Defense

- a) (6 points) Why does an attacker need to bother with XSS? Wouldn't it be much easier to just create a malicious page with a script that steals all cookies of all pages from the user's browser? (Hint: Name the setting used by browser to defend against such attacks.)

The reason:

- b) (8 points) Given the server side code *hw2-xss.php*, perform an XSS attack to steal a user's cookies. Some instructions to set up the environment are as follows.

- You should install PHP to execute PHP code.
- In the directory of *hw2-xss.php*, run command `php -S localhost:8000` to start a server on your localhost with port 8000.
- Visit the server in your browser with the URL `http://localhost:8000/hw2-xss.php?name=xxx`
- You are not allowed to modify *hw2-xss.php*. You need to specify the parameter *name* in your URL.

You should perform an XSS attack in your browser with some crafted URLs by injecting Javascript code. What is the name field in this crafted URL? (Hint: You should use the function `alert()` to pop a message box.) You should attach a screenshot and rename it to *q3b.jpg* to demonstrate a successful exploit.

name=

- c) (7 points) The developer decides to fix this issue by removing all instances of `<script>` and `</script>` tags when parsing the URLs on the server side. For example, `<script>code</script>` turns to be `code`. Note that the removal process removes all `<script>` and `</script>` pairs that can be recognized as HTML tags, including inserting spaces in the tags between the bracket and the text, for example, `<script >code< /script>`. Please provide an attack under this condition.

name=

- d) (5 points) Name a method that can fix this kind of XSS vulnerabilities.

5. (24 points) **CSRF Attacks and Defense.**

- a) (6 points) Attackers can leverage XSS vulnerabilities to launch CSRF attacks to force an end user to execute unwanted actions. Apart from XSS, name another possible way that attacker can use to launch CSRF attacks and provide necessary explanation.

- b) (10 points) Use *hw2-xss.php* and *index.html* to perform a CSRF attack.

- You are acting as the attacker who controls a server hosting the file *index.html*, while your browser acts as the victim to visit *index.html*. Your task is to perform a CSRF attack by visiting *hw2-xss.php* from *index.html*.

- First start the PHP server using the same command in the question above to set the *hw2-xss.php* under `localhost:8000`.
- Start another server to host the file *index.html*. To create a different domain locally, you need to modify system *hosts* file in your machine by adding a new map of hostname and IP address. Specifically, you need to add a line of `127.0.0.1 malicious-server` to the *hosts* file to map hostname `malicious-server` to IP address `127.0.0.1`.
Generally, for Unix/Linux systems, the *hosts* file is located at `/etc/hosts`, and for Windows systems, it is at `C:\Windows\System32\drivers\etc\hosts`.
After that, you can use the command below in the directory of *index.html* to start another server on `malicious-server:8002`

```
python3 -m http.server -b=malicious-server 8002
```


Now you can visit the server by URL <http://malicious-server:8002/index.html>.
- You are expected to see responses from both the terminal of *index.html* and the terminal hosting *hw2-q4-xss.php*.

What is the response you get from each server? Copy and paste the one with HTTP Status Code.

For *index.html*, it looks like `$IP - - [$time] "$command"$status-code -`.

For *hw2-xss.php*, it looks like `[$time] $IP [$status-code]: "$command"`.

You should attach a screenshot of your visit to <http://malicious-server:8002/index.html> and rename it to *q4b.jpg*.

- c) (8 points) If you directly forbid cross-site requests in the browser, are you immune from XSS or CSRF? Select yes or no for XSS and CSRF, respectively.

For XSS: Yes ☐ No ☐

For CSRF: Yes ☐ No ☐